**SHETH L.U.J AND SIR M.V. COLLEGE**

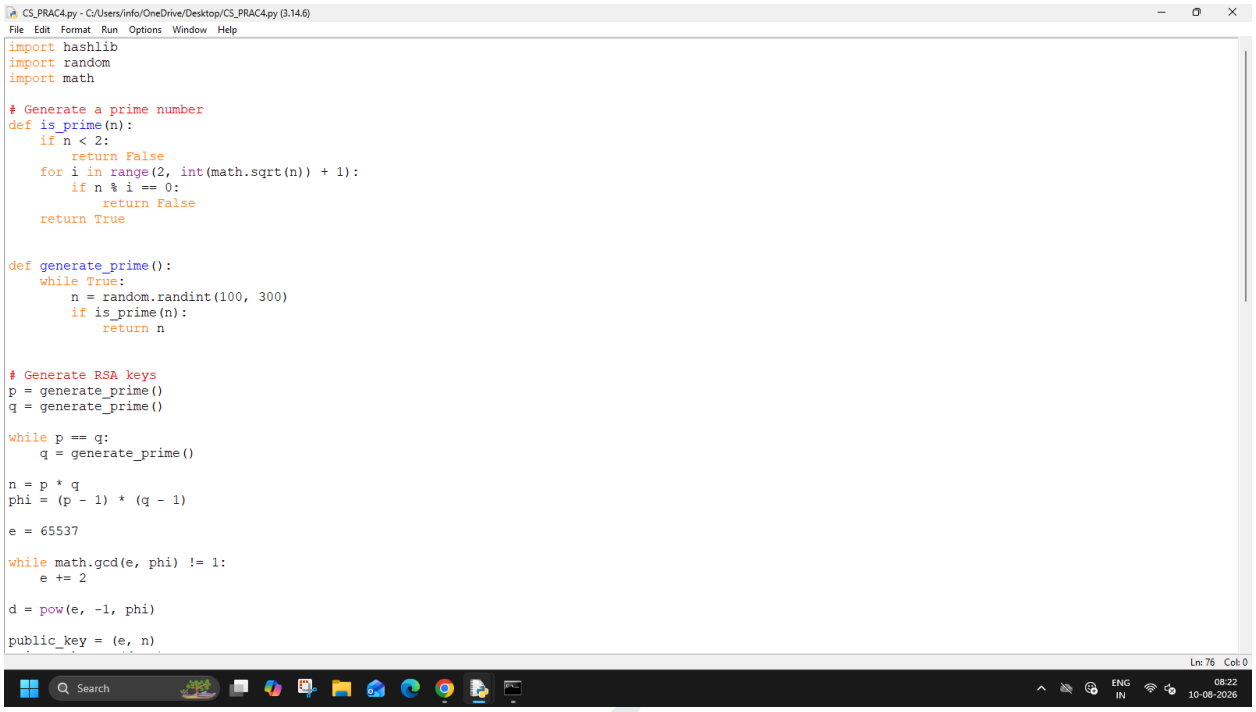
SUBJECT:CYBER & INFORMATION SECURITY

**PRACTICAL - 4**

**AIM:**Implement digital signature algorithms such as RSA-based signatures, and verify the integrity and authenticity of digitally signed messages.

**CLI:**

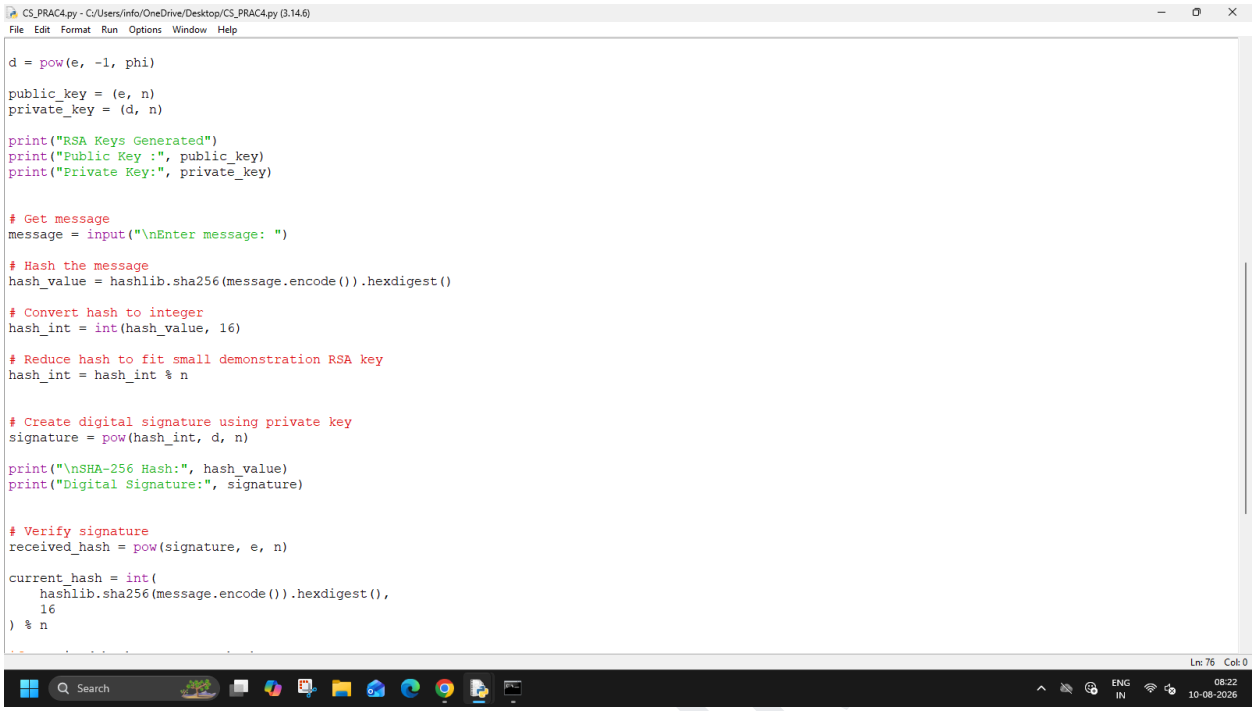
**Code:**

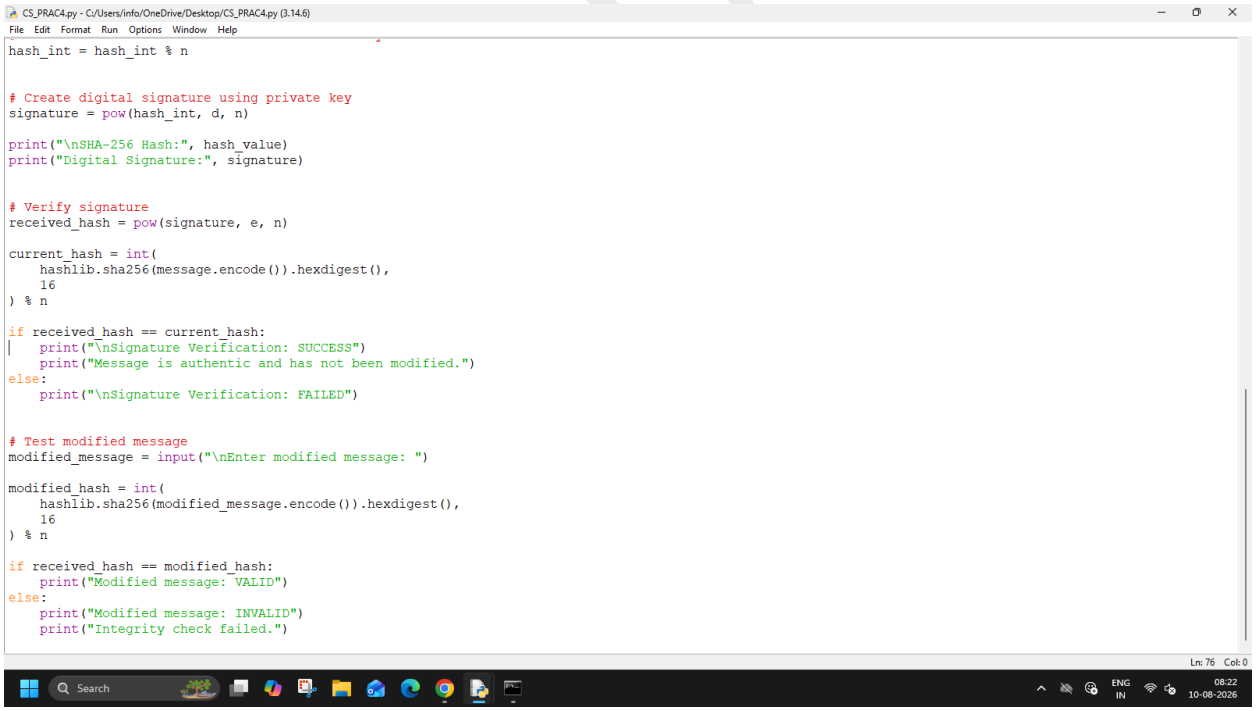





**SUMEET YADAV**

**T125**



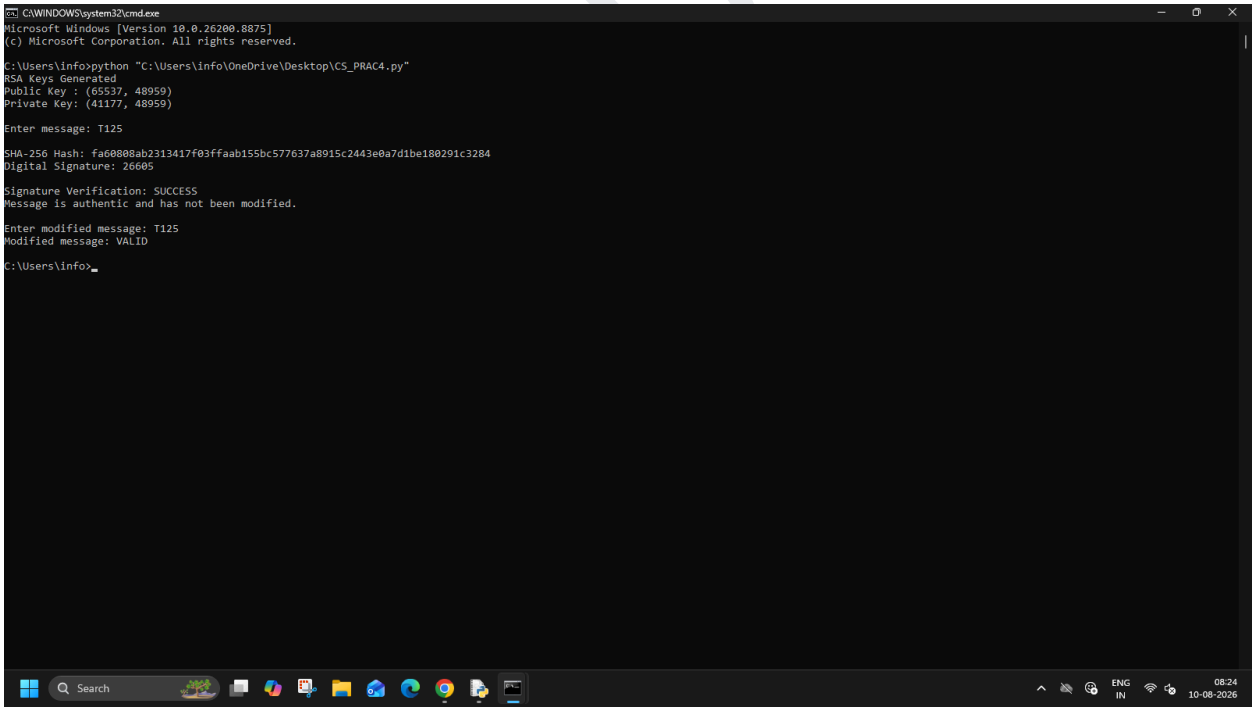




**SUMEET YADAV**
**T125**

**SHETH L.U.J AND SIR M.V. COLLEGE**

SUBJECT:CYBER & INFORMATION SECURITY

## OUTPUT:


**SUMEET YADAV**

**T125**

**SHETH L.U.J AND SIR M.V. COLLEGE**

SUBJECT:CYBER & INFORMATION SECURITY

**GUI:**

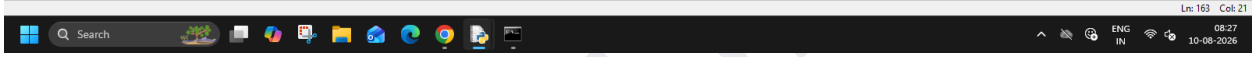
**Code:**

*cs_prac5_gui.py - C:/Users/info/OneDrive/Desktop/cs_prac5_gui.py (3.14.6)*

File Edit Format Run Options Window Help

```
import tkinter as tk
from tkinter import messagebox
import hashlib
import random
import math

#RSA FUNCTIONS
def is_prime(n):
    if n < 2:
        return False

    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

def generate_prime():
    while True:
        number = random.randint(100, 300)

        if is_prime(number):
            return number

def generate_keys():
    global public_key, private_key

    p = generate_prime()
    q = generate_prime()
    while p == q:
        q = generate_prime()
    n = p * q
    phi = (p - 1) * (q - 1)
    e = 65537
    while math.gcd(e, phi) != 1:
        e += 2
    d = pow(e, -1, phi)
    public_key = (e, n)
    private_key = (d, n)
    public_key_label.config(
```

Ln: 163 Col: 21



*cs_prac5_gui.py - C:/Users/info/OneDrive/Desktop/cs_prac5_gui.py (3.14.6)*

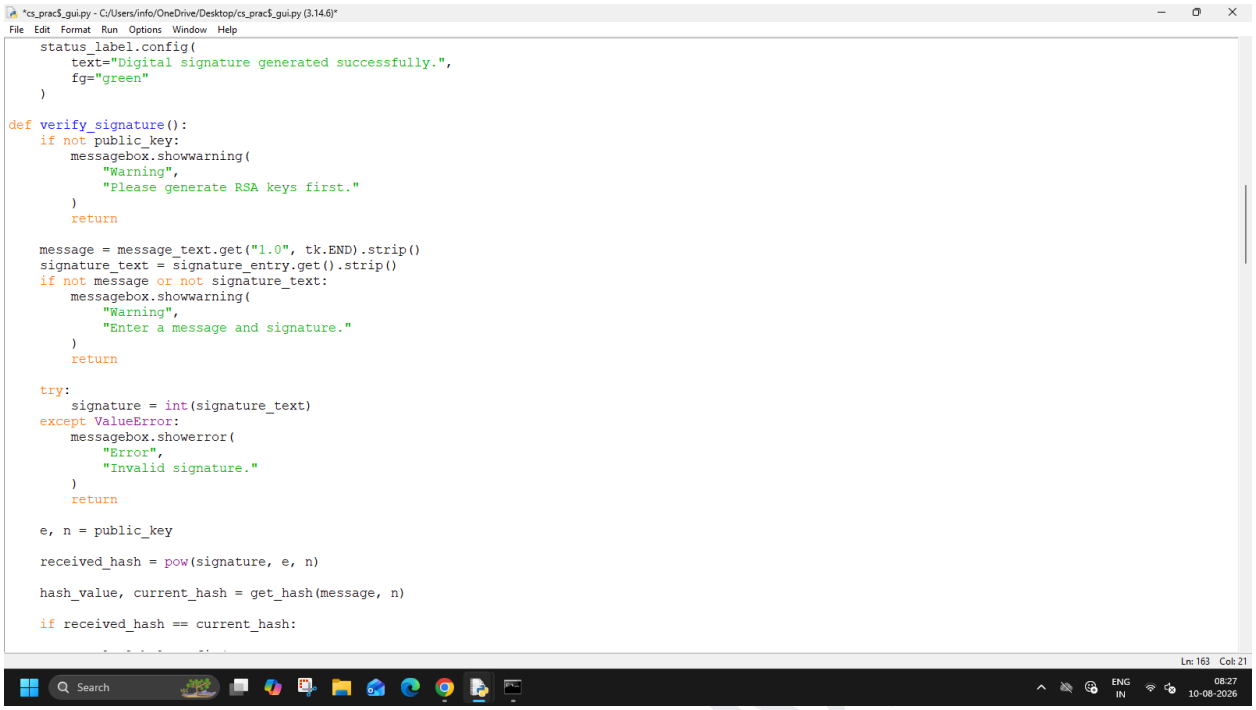
File Edit Format Run Options Window Help

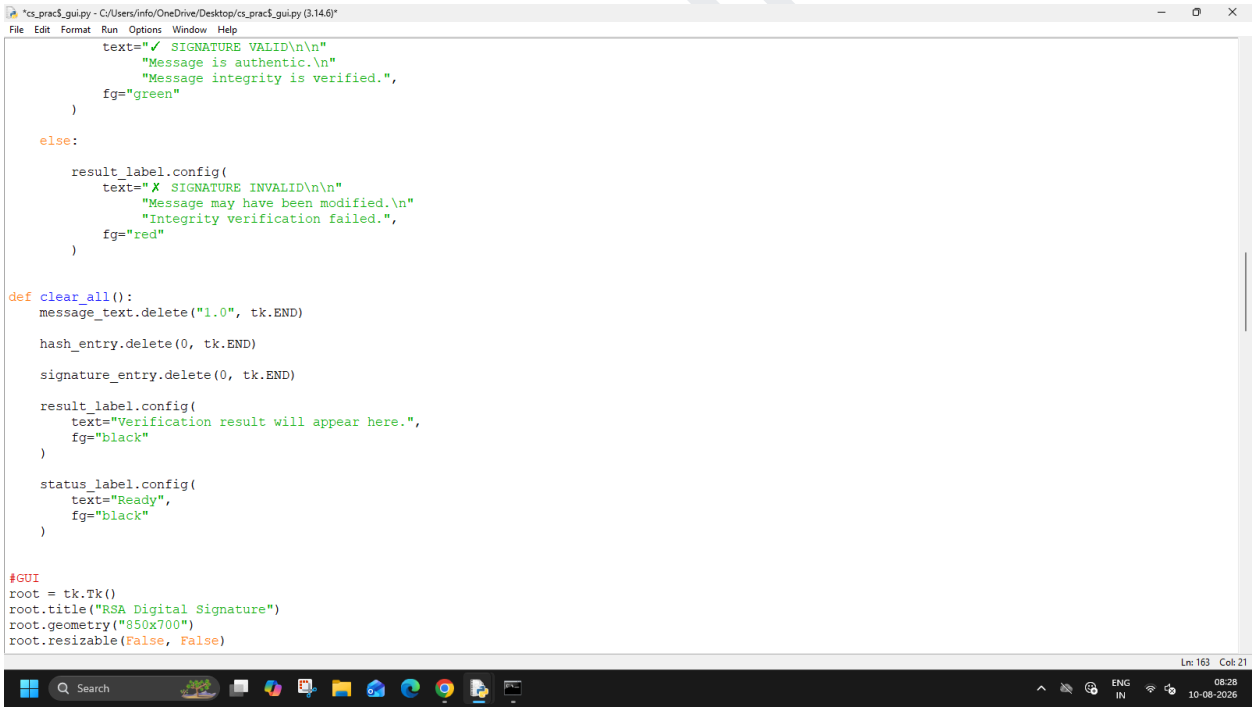
```
    status_label.config(
        text="RSA keys generated successfully.",
        fg="green"
    )

def get_hash(message, n):
    hash_value = hashlib.sha256(
        message.encode()
    ).hexdigest()
    hash_integer = int(hash_value, 16)
    return hash_value, hash_integer % n

def sign_message():
    if not public_key or not private_key:
        messagebox.showwarning(
            "Warning",
            "Please generate RSA keys first."
        )
        return
    message = message_text.get("1.0", tk.END).strip()
    if not message:
        messagebox.showwarning(
            "Warning",
            "Please enter a message."
        )
        return

    d, n = private_key
    hash_value, hash_integer = get_hash(message, n)
    signature = pow(hash_integer, d, n)
    hash_entry.delete(0, tk.END)
    hash_entry.insert(0, hash_value)
    signature_entry.delete(0, tk.END)
    signature_entry.insert(0, str(signature))

    status_label.config(
        text="Digital signature generated successfully.",
        fg="green"
    )
```

Ln: 163 Col: 21

**SUMEET YADAV**

**T125**




**SUMEET YADAV**
**T125**

*cs_prac5_gui.py - C:/Users/info/OneDrive/Desktop/cs_prac5_gui.py (3.14.6)*

File Edit Format Run Options Window Help

```
# Header
header = tk.Frame(root, bg="#1f2937", height=80)
header.pack(fill="x")
title = tk.Label(
    header,
    text="RSA DIGITAL SIGNATURE",
    font=("Arial", 24, "bold"),
    bg="#1f2937",
    fg="white"
)
title.pack(pady=20)

subtitle = tk.Label(
    root,
    text="Cyber & Information Security | MU NEP 2020 - Semester 5",
    font=("Arial", 11),
    fg="#555555"
)
subtitle.pack(pady=10)


# Key section
key_frame = tk.LabelFrame(
    root,
    text="RSA Key Generation",
    font=("Arial", 11, "bold"),
    padx=15,
    pady=10
)
key_frame.pack(fill="x", padx=25, pady=5)

generate_button = tk.Button(
    key_frame,
    text="GENERATE RSA KEYS",
    command=generate_keys,
    font=("Arial", 11, "bold"),
    padx=15,
    pady=8
)
```

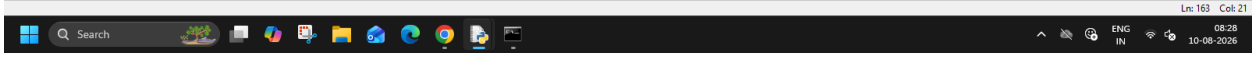
Ln: 163 Col: 21


*cs_prac5_gui.py - C:/Users/info/OneDrive/Desktop/cs_prac5_gui.py (3.14.6)*

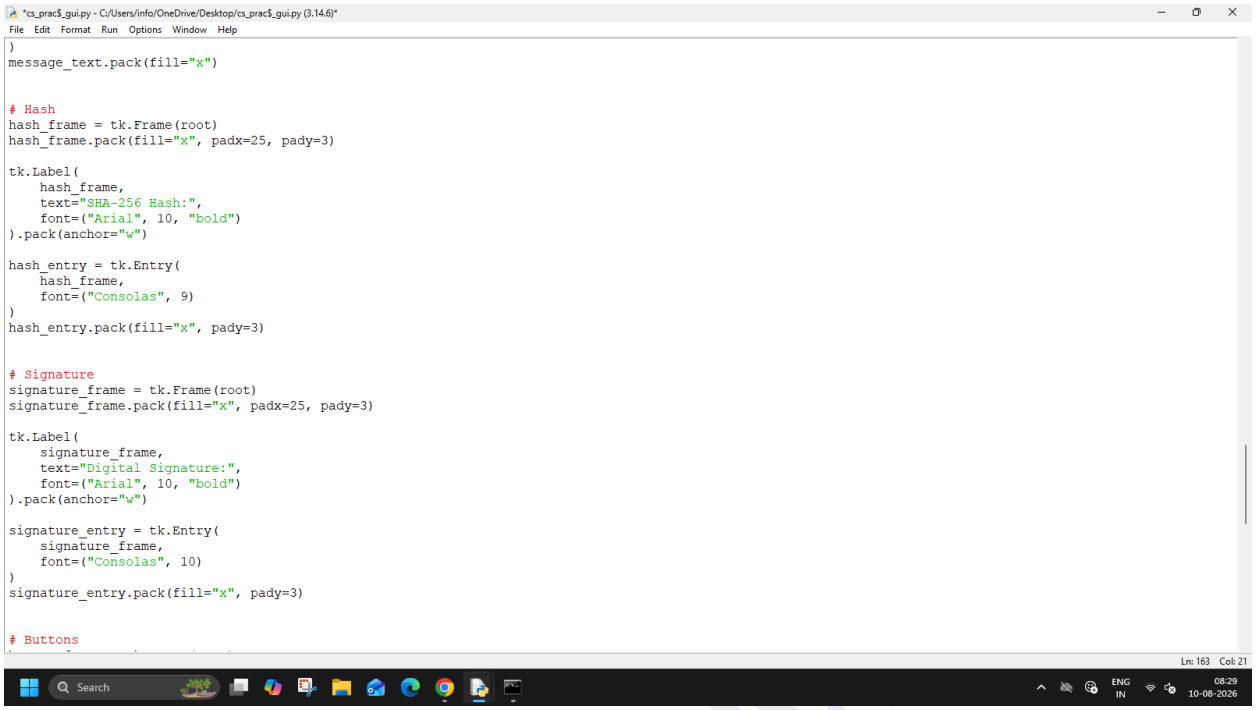
File Edit Format Run Options Window Help

```
public_key_label = tk.Label(
    key_frame,
    text="Public Key: Not generated",
    anchor="w",
    justify="left",
    font=("Consolas", 9)
)
public_key_label.pack(fill="x", pady=5)

private_key_label = tk.Label(
    key_frame,
    text="Private Key: Not generated",
    anchor="w",
    justify="left",
    font=("Consolas", 9)
)
private_key_label.pack(fill="x")


# Message section
message_frame = tk.LabelFrame(
    root,
    text="Message",
    font=("Arial", 11, "bold"),
    padx=10,
    pady=10
)
message_frame.pack(fill="x", padx=25, pady=10)

message_text = tk.Text(
    message_frame,
    height=4,
    font=("Arial", 11),
    wrap="word"
)
message_text.pack(fill="x")
```

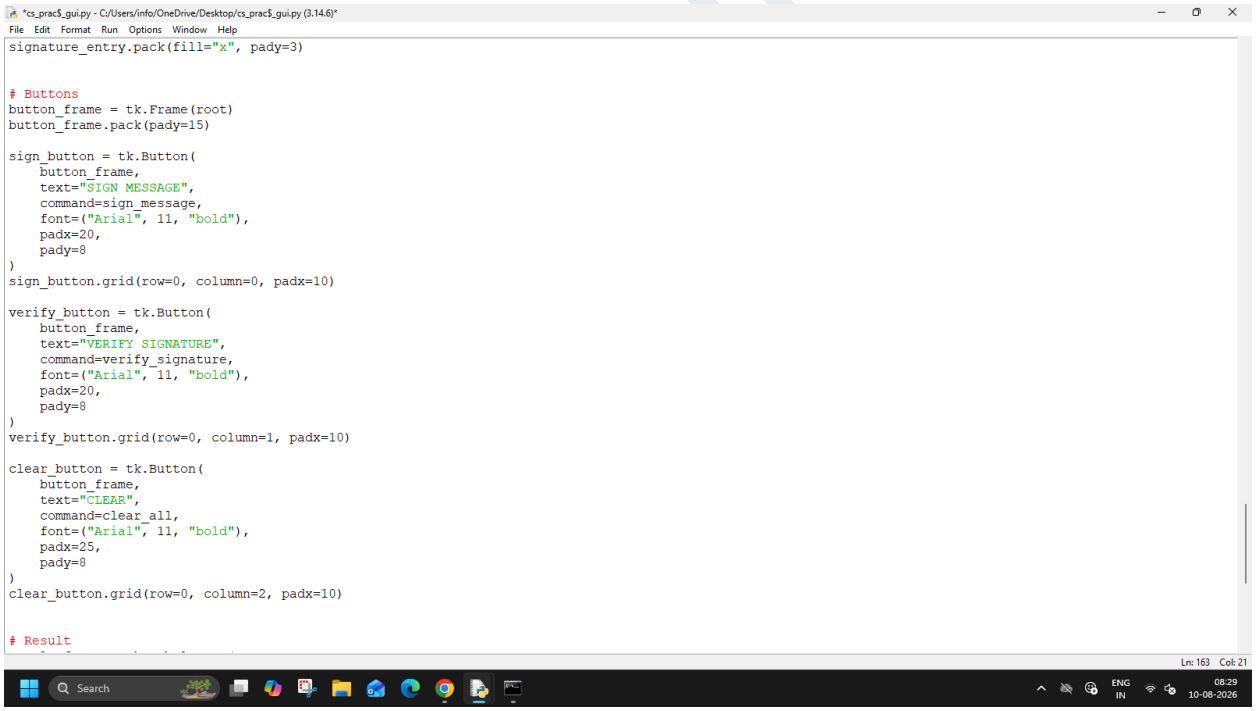
Ln: 163 Col: 21




**SUMEET YADAV**
**T125**



```
)
message_text.pack(fill="x")


# Hash
hash_frame = tk.Frame(root)
hash_frame.pack(fill="x", padx=25, pady=3)

tk.Label(
    hash_frame,
    text="SHA-256 Hash:",
    font=("Arial", 10, "bold")
).pack(anchor="w")

hash_entry = tk.Entry(
    hash_frame,
    font=("Consolas", 9)
)
hash_entry.pack(fill="x", pady=3)


# Signature
signature_frame = tk.Frame(root)
signature_frame.pack(fill="x", padx=25, pady=3)

tk.Label(
    signature_frame,
    text="Digital Signature:",
    font=("Arial", 10, "bold")
).pack(anchor="w")

signature_entry = tk.Entry(
    signature_frame,
    font=("Consolas", 10)
)
signature_entry.pack(fill="x", pady=3)


# Buttons
```



```
signature_entry.pack(fill="x", pady=3)


# Buttons
button_frame = tk.Frame(root)
button_frame.pack(pady=15)

sign_button = tk.Button(
    button_frame,
    text="SIGN MESSAGE",
    command=sign_message,
    font=("Arial", 11, "bold"),
    padx=20,
    pady=8
)
sign_button.grid(row=0, column=0, padx=10)

verify_button = tk.Button(
    button_frame,
    text="VERIFY SIGNATURE",
    command=verify_signature,
    font=("Arial", 11, "bold"),
    padx=20,
    pady=8
)
verify_button.grid(row=0, column=1, padx=10)

clear_button = tk.Button(
    button_frame,
    text="CLEAR",
    command=clear_all,
    font=("Arial", 11, "bold"),
    padx=25,
    pady=8
)
clear_button.grid(row=0, column=2, padx=10)


# Result
```

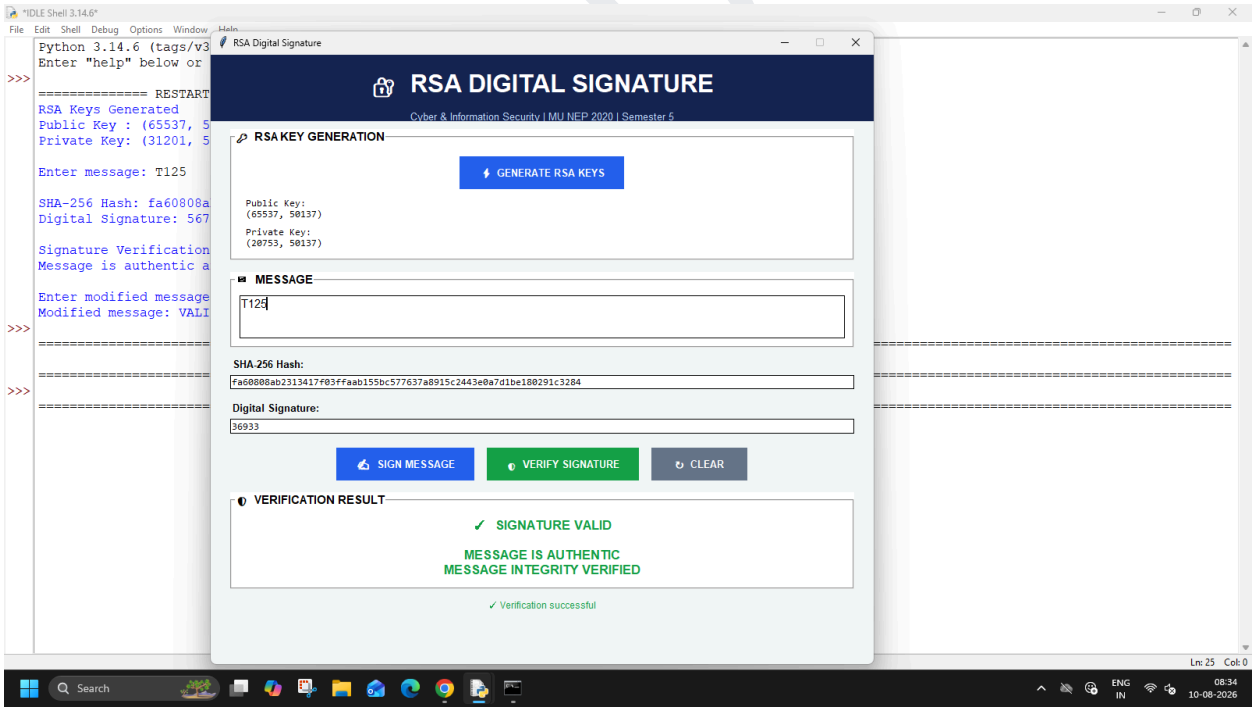
**SUMEET YADAV**
**T125**

**SHETH L.U.J AND SIR M.V. COLLEGE**

SUBJECT:CYBER & INFORMATION SECURITY

```
)
clear_button.grid(row=0, column=2, padx=10)


# Result
result_frame = tk.LabelFrame(
    root,
    text="Verification Result",
    font=("Arial", 11, "bold"),
    padx=10,
    pady=10
)
result_frame.pack(fill="x", padx=25, pady=5)

result_label = tk.Label(
    result_frame,
    text="Verification result will appear here.",
    font=("Arial", 12, "bold"),
    justify="center"
)
result_label.pack(pady=10)


# Status
status_label = tk.Label(
    root,
    text="Ready",
    font=("Arial", 10)
)
status_label.pack(pady=8)


# Variables
public_key = None
private_key = None


root.mainloop()
```

## OUTPUT:




**SUMEET YADAV**

**T125**